Buildmart — Speaker Brief

Capstone B Sprint 4 · Group 2 · 21 slides presented, 14 min 55 sec · Slides 22–26 are backup, not presented

Ranjan	**Slides 1–4**	Opening, the problem, users and objectives
Sujan	**Slides 5–8**	Requirements, sprint journey, GitHub, team
Kalyan	**Slides 9–10**	Database decisions and the ERD
Diya	**Slide 11**	Front-end development
Shailendra	**Slides 12–13**	Back end and the integration flow
Dipesh	**Slides 14–16**	CRUD and checkout
Anjan	**Slides 17–18**	Testing and validation
Diya + Kalyan	**Slide 19**	Live demonstration — Diya clicks, Kalyan narrates
Anjan	**Slide 20**	Reflection and future improvements
Ranjan	**Slide 21**	Conclusion, then questions

Ranjan — slides 1–4 and 21

2 min 00 sec

1. Title — 15s

- Name the project, the team, and that it is Sprint 4. Nothing more.
- The four numbers (8 · 19 · 100 · 86) are proof points — let them sit, do not read them.

2. What we will cover — 15s

- Read the *names*, not the descriptions. This tells the marker who owns what before anyone speaks.

3. The problem we set out to solve — 45s

- Lead with the business problem, not the technology.
- Before: product data lived in the page, the cart was lost on refresh, nothing stopped over-ordering.
- Say the bottom line out loud: *"The front end never decides what is true. It asks the API, and the API asks the database."*

4. Target users and objectives — 25s

- Retail customers, returning customers, and store staff in future.
- Do not read all six features — say "catalogue, cart and accounts" and move on.

21. Conclusion — 20s

- Do not read the eight boxes. One sentence: four static pages became a working three-tier system.
- Then invite questions and direct each one to whoever presented that slide.

Your Q&A: Q1 what problem it solves · Q2 who the users are · Q3 the main features

Sujan — slides 5–8

2 min 15 sec

5. Requirements analysis — 40s

- Functional and non-functional down the left; the business rules table on the right is the strongest part.
- Every rule names *where it is enforced* — the rules live in the system, not in a document.

6. The sprint journey — 30s

- Four sprints: static site → cart in the browser → database and API → integration and accounts.
- Be honest that Sprint 4 was mostly integration. Markers reward an accurate account.

7. GitHub evidence — 35s

- 86 commits · 21 merged pull requests · 7 members · 2 CI workflows.
- Issue → branch → pull request → review → CI. Have the Actions tab open showing green.

8. Team contributions — 30s

- One line per member. Do not read the table.
- Say the caveat: commit counts record participation, not value — one integration commit can be a week of work.

Your Q&A: Q23 how GitHub supported teamwork · Q24 how work was divided

Kalyan — slides 9–10, and narrating the demo

1 min 40 sec + demo

9. Database design decisions — 40s

- **Why SQL:** a cart line is meaningless without its cart and product. Foreign keys, UNIQUE and CHECK do the work.
- **Why SQLite:** one file, zero configuration, and each test file makes its own temporary database.
- **Say the trade-off unprompted:** SQLite handles one writer at a time — fine for one store, a bigger deployment moves to MySQL and the schema ports directly.

10. Entity relationship diagram — 60s *(worth 5 marks — the joint highest)*

- Eight entities: three for the shop, two for orders, three for accounts.
- `cart_items` is the junction table resolving carts-to-products; `order_items` does the same for orders.
- **The point that impresses:** `order_items` copies product_name and unit_price, so an order keeps the price actually paid even if the product price changes later.

Your Q&A: **Q4** how many entities → **eight** · **Q5** primary key → uniquely identifies a row, e.g. product_id · **Q6** foreign key → points at another table's primary key, e.g. cart_items.cart_id (add: we switch it on with PRAGMA foreign_keys = ON) · **Q7** one relationship → carts to cart_items, one-to-many · **Q12** why this stack

Diya — slide 11, and driving the demo

0 min 40 sec + demo

11. Front-end development — 40s

- Plain HTML, CSS and JavaScript. No framework. Seven pages, one stylesheet.
- **The key idea is the "does / does not" pair:** the front end renders what the API returns. It does not hold the cart, calculate the total, or decide what is in stock.
- Registration form validates live — but the server re-checks everything, because a browser can be bypassed.

Your Q&A: Q15 how information is displayed · Q16 what happens when a user submits something

Shailendra — slides 12–13

1 min 50 sec

12. Back-end development — 50s

- Node.js, Express, better-sqlite3, bcryptjs, jsonwebtoken, node:test.
- Six duties: validate, authorise, enforce rules, query, calculate, respond.
- Security: the JWT secret comes from the environment; login gives the same message for a wrong password and an unknown email, so nobody can discover which emails have accounts.

13. How information flows — 60s *(the brief calls this the most important section)*

- Trace the arrow top to bottom with your finger: user → front end → API → database → response → page.
- Use the real example: clicking **+** sends PATCH, the server validates and checks stock, SQL updates the row, a JOIN recalculates, the drawer redraws at $124.75 with GST $11.34.

Your Q&A: Q9 role of the backend · Q10 how it connects to the database (better-sqlite3, prepared statements) · Q11 what endpoints exist · Q13 Q14 how a click reaches the data

Dipesh — slides 14–16

2 min 05 sec

14. CRUD — Create and Read — 40s

- The green blocks are real responses captured from the running server. Say so.
- Adding the same product twice raises the quantity instead of making a second row — that is the UNIQUE constraint.

15. CRUD — Update and Delete — 40s

- Make the distinction: **−** lowers the quantity by one via PATCH; only reaching zero deletes. **Remove** deletes the whole line whatever the quantity.

16. Checkout — cart becomes a real order — 45s

- One click writes three things: an `orders` row, `order_items` with the prices paid, and the stock coming down.
- **Say the transaction point:** if the stock check fails on the second product, the first product's stock is not taken and no order survives. All or nothing.

Your Q&A: Q17 how CRUD was tested

Anjan — slides 17–18 and 20

2 min 05 sec

17. Testing evidence — 45s

- Lead with **100 of 100 passing**. 45 auth, 23 orders, 20 cart, 7 static, 5 seed.
- Each test file builds its own temporary database, so tests never touch the real one — that is why they can be run repeatedly.

18. Validation and error handling — 35s

- 400 for bad input · 404 for unknown product, cart or line · 409 over stock · 401 for bad credentials.
- **Make the security point:** a wrong password and an unknown email return the identical 401, so an attacker cannot discover which emails have accounts.

20. Reflection and future improvements — 45s

- Hardest problem: connecting front end to back end — CORS errors and totals not refreshing. Solved by serving the pages from the same origin and redrawing from the API response.
- Name the three limitations before a marker finds them: no payment or email, guest cart not merged on login, no staff role.
- **Do not say** carts are unlinked, orders are missing, or stock does not drop — all three are now built, and the demo shows them.

Your Q&A: Q18 bugs found · Q19 how they were fixed · Q20 invalid input · Q25–Q28 future work and security

Slide 19 — Live demonstration

2 min 20 sec · Diya clicks, Kalyan narrates

Setup, ten minutes before

- Window 1: `cd backend` → `npm run reset-db` → `npm start`, then leave it alone.
- Open `http://localhost:3000/shop.html` — not the file, or the products will not load.
- Register the demo account now, so step 2 is a login.
- DB Browser open on `buildmart.db` beside the browser. **Press F5 after every click.**

The eight steps

- **1 Read** — open the shop. “Eight products, read from SQLite.”
- **2 Accounts** — log in. “The cart now belongs to this account.”
- **3 Create** — add two products. “Two new rows in cart_items.”
- **4 Update** — click +. “A PATCH went to the API; the total came back recalculated.”
- **5 Delete** — click Remove. “The row is gone. Not hidden — deleted.”
- **6 Validate** — add Safety Glasses four times. “Stock is three; the server refused the fourth with a 409.”
- **7 Checkout** — **the moment that earns the marks.** Show orders, then order_items, then products. “One click, three tables, one transaction.” Go slowly.
- **8 Read** — My Orders. “The same order, read back from the API.”

If it breaks: say “let me reset”, run `npm run reset-db`, carry on. Never debug in front of the marker. Never click “Forgot password?” — it is a Version 2 placeholder.

Rules for everyone

- Only the presenter of a slide speaks on it. Save additions for Q&A.
- Hand over by name — “Sujan will take you through our planning.”
- Q&A goes to whoever presented that slide.
- **Q21 and Q22 are personal** — your own contribution and your own hardest task. Everyone needs their own answer.
- Clock check at slide 13. Later than seven minutes in, speed up — the demo keeps its full 2 min 20.